Tree Based Methods

IS382 - Predictive Modeling

Jason Bryer

Week 7

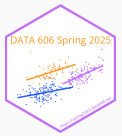
Classification and Regression Trees

The goal of CART methods is to find best predictor in X of some outcome, y. CART methods do this recursively using the following procedures:

- Find the best predictor in X for y.
- Split the data into two based upon that predictor.
- Repeat 1 and 2 with the split data sets until a stopping criteria has been reached.

There are a number of possible stopping criteria including: Only one data point remains.

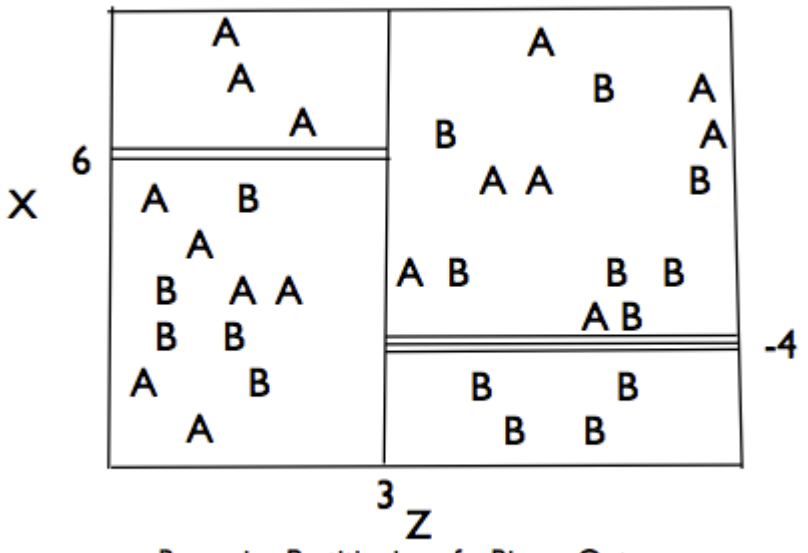
- All data points have the same outcome value.
- No predictor can be found that sufficiently splits the data.



Recursive Partitioning Logic of CART

Consider the scatter plot to the right with the following characteristics:

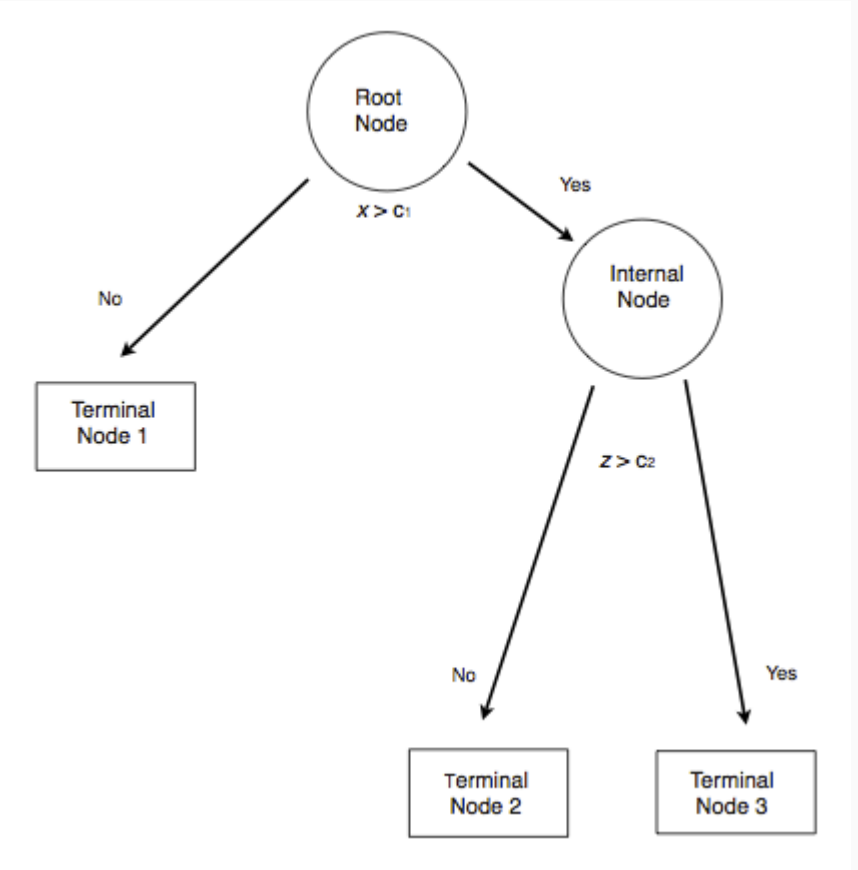
- Binary outcome, G, coded “A” or “B”.
- Two predictors, x and z
- The vertical line at z = 3 creates the first partition.
- The double horizontal line at x = -4 creates the second partition.
- The triple horizontal line at x = 6 creates the third partition.



Recursive Partitioning of a Binary Outcome
(where G = A or B and predictors are Z and X)

Tree Structure

- The root node contains the full data set.
- The data are split into two mutually exclusive pieces. Cases where x > ci go to the right, cases where x <= ci go to the left.
- Those that go to the left reach a terminal node.
- Those on the right are split into two mutually exclusive pieces. Cases where z > c2 go to the right and terminal node 3; cases where z <= c2 go to the left and terminal node 2.



Sum of Squared Errors

The sum of squared errors for a tree *T* is:

$$S = \sum_{c \in leaves(T)} \sum_{i \in c} (y - m_c)^2$$

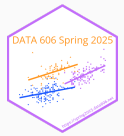
Where, $m_c = \frac{1}{n} \sum_{i \in c} y_i$, the prediction for leaf \textit{c}.

Or, alternatively written as:

$$S = \sum_{c \in leaves(T)} n_c V_c$$

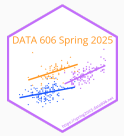
Where V_c is the within-leave variance of leaf \textit{c}.

Our goal then is to find splits that minimize S.



Advantages of CART Methods

- Making predictions is fast.
- It is easy to understand what variables are important in making predictions.
- Trees can be grown with data containing missingness. For rows where we cannot reach a leaf node, we can still make a prediction by averaging the leaves in the sub-tree we do reach.
- The resulting model will inherently include interaction effects. There are many reliable algorithms available.

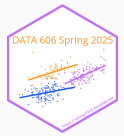


Regression Trees

In this example we will predict the median California house price from the house's longitude and latitude.

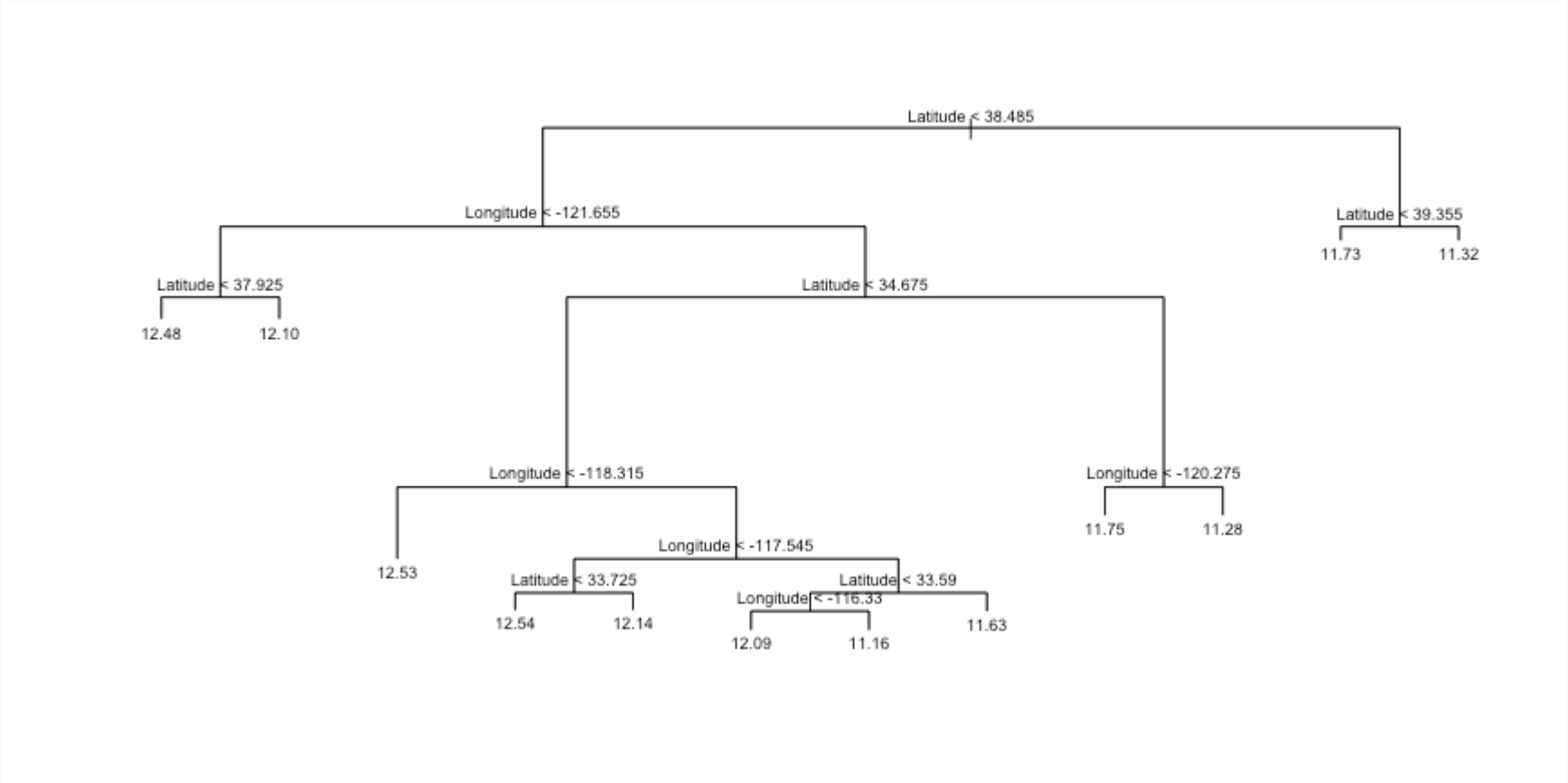
```
str(calif)
```

```
## 'data.frame':    20640 obs. of  10 variables:
##  $ MedianHouseValue: num  452600 358500 352100 341300 342200 ...
##  $ MedianIncome    : num  8.33 8.3 7.26 5.64 3.85 ...
##  $ MedianHouseAge  : num  41 21 52 52 52 52 52 52 42 52 ...
##  $ TotalRooms      : num  880 7099 1467 1274 1627 ...
##  $ TotalBedrooms   : num  129 1106 190 235 280 ...
##  $ Population      : num  322 2401 496 558 565 ...
##  $ Households      : num  126 1138 177 219 259 ...
##  $ Latitude        : num  37.9 37.9 37.9 37.9 37.9 ...
##  $ Longitude       : num  -122 -122 -122 -122 -122 ...
##  $ cut.prices      : Factor w/ 4 levels "[1.5e+04,1.2e+05]",..: 4 4 4 4 4 4 4 3 3 3 ...
```

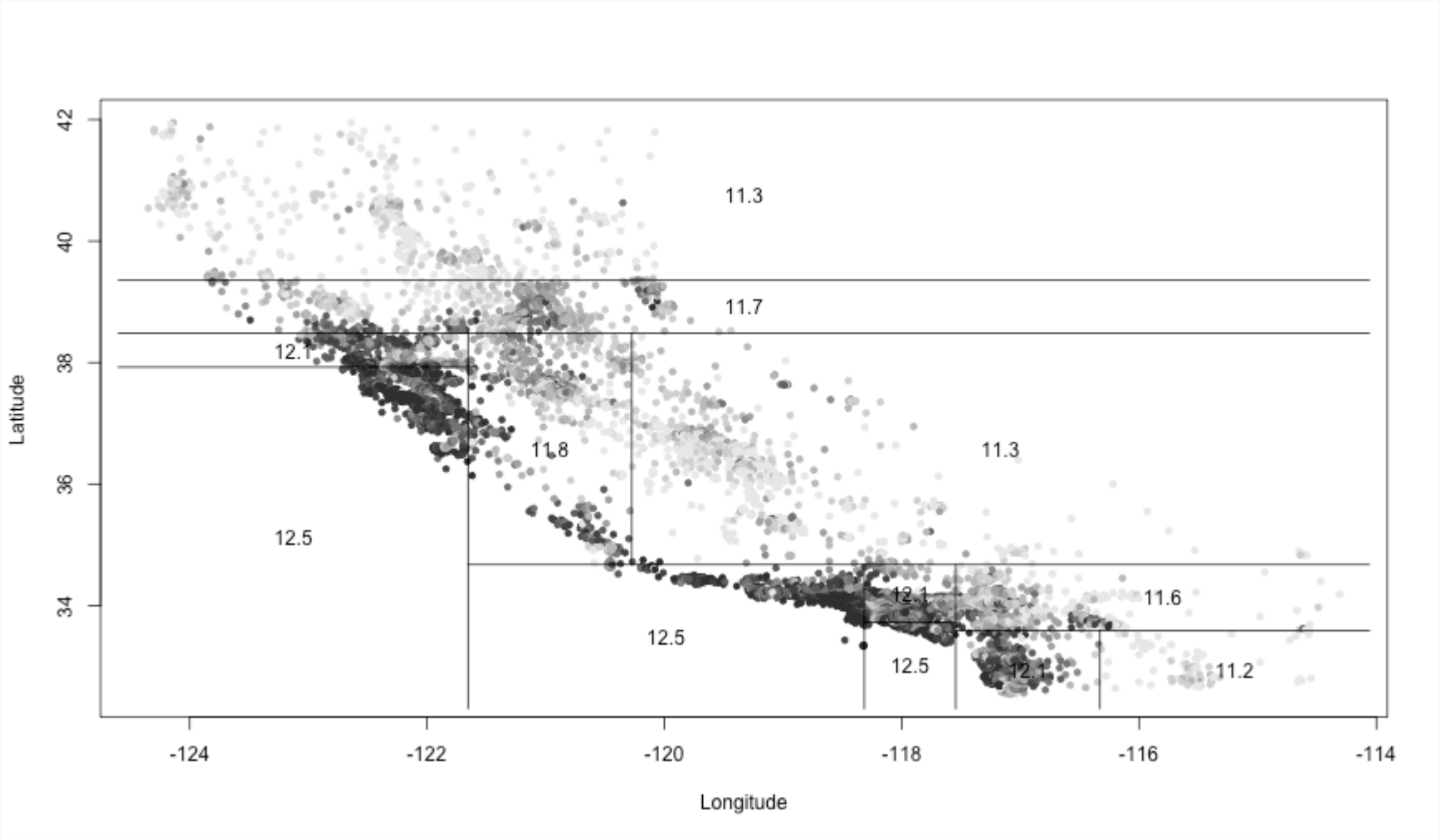


Tree 1

```
treefit <- tree(log(MedianHouseValue) ~ Longitude + Latitude, data=calif)
plot(treefit); text(treefit, cex=0.75)
```



Tree 1

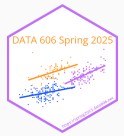

Latitude
42
40
38
36
34
11.3
11.7
12.1
11.8
11.3
12.5
12.1
11.6
12.5
12.5
12.1
11.2
-124
-122
-120
-118
-116
-114
Longitude

Tree 1

```
summary(treefit)
```

```
##
## Regression tree:
## tree(formula = log(MedianHouseValue) ~ Longitude + Latitude,
##     data = calif)
## Number of terminal nodes:  12
## Residual mean deviance:  0.1662 = 3429 / 20630
## Distribution of residuals:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -2.75900 -0.26080 -0.01359  0.00000  0.26310  1.84100
```

Here “deviance” is the mean squared error, or root-mean-square error of $\sqrt{.166} = 0.41$.



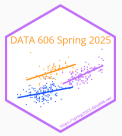
Tree 2, Reduce Minimum Deviance

We can increase the fit but changing the stopping criteria with the mindev parameter.

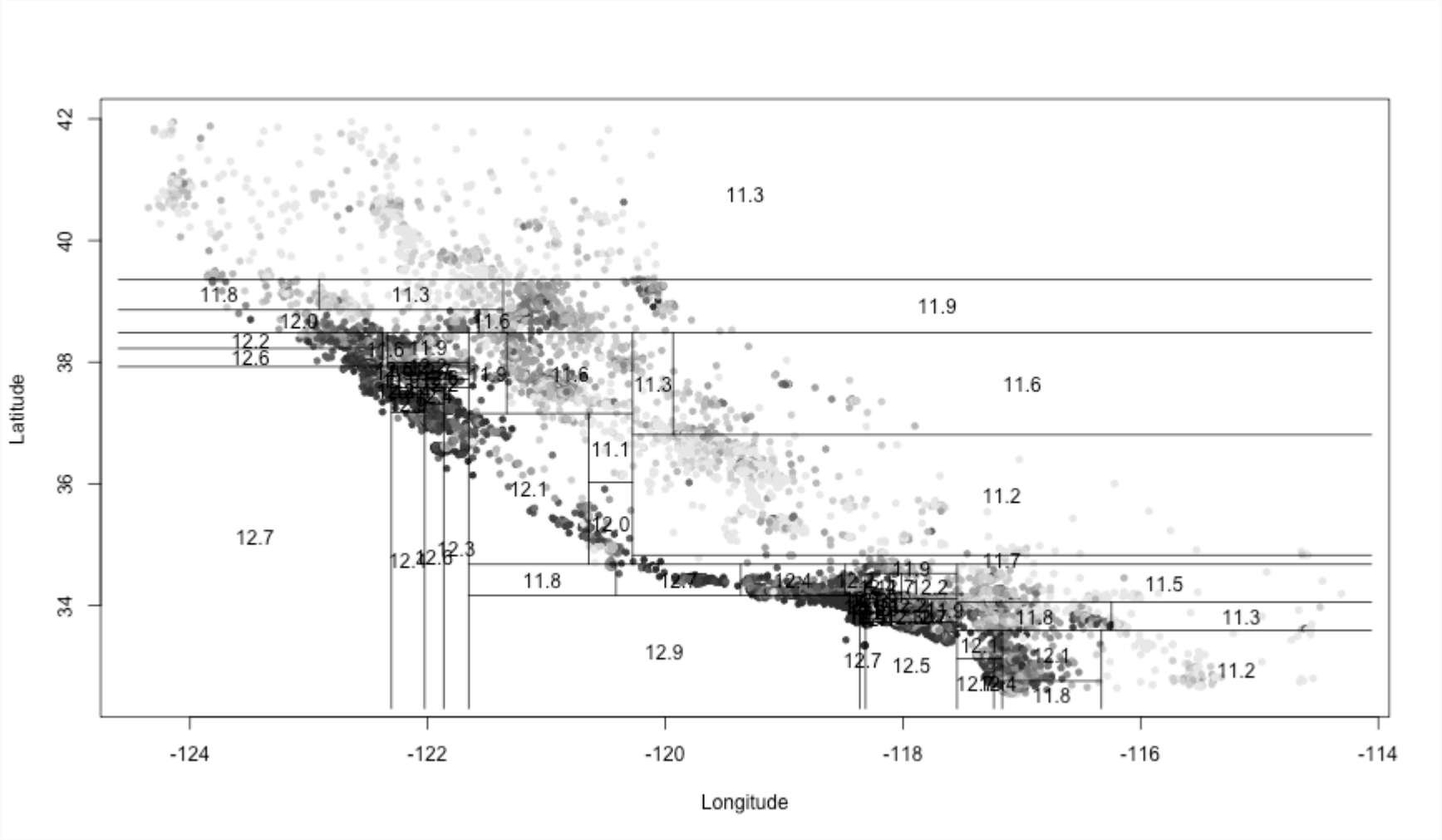
```
treefit2 <- tree(log(MedianHouseValue) ~ Longitude + Latitude, data=calif, mindev=.001)
summary(treefit2)
```

```
##
## Regression tree:
## tree(formula = log(MedianHouseValue) ~ Longitude + Latitude,
##     data = calif, mindev = 0.001)
## Number of terminal nodes:  68
## Residual mean deviance:  0.1052 = 2164 / 20570
## Distribution of residuals:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -2.94700 -0.19790 -0.01872  0.00000  0.19970  1.60600
```

With the larger tree we now have a root-mean-square error of 0.32.



Tree 2, Reduce Minimum Deviance



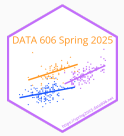
Tree 3, Include All Variables

However, we can get a better fitting model by including the other variables.

```
treefit3 <- tree(log(MedianHouseValue) ~ ., data=calif)
summary(treefit3)
```

```
##
## Regression tree:
## tree(formula = log(MedianHouseValue) ~ ., data = calif)
## Variables actually used in tree construction:
## [1] "cut.prices"
## Number of terminal nodes:  4
## Residual mean deviance:  0.03608 = 744.5 / 20640
## Distribution of residuals:
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -1.718000 -0.127300  0.009245  0.000000  0.130000  0.358600
```

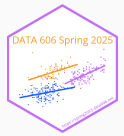
With all the available variables, the root-mean-square error is 0.11.



Classification Trees

Predicting who survived the Titanic.

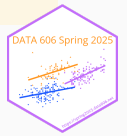
- `pclass` : Passenger class (1 = 1st; 2 = 2nd; 3 = 3rd)
- `survival` : A Boolean indicating whether the passenger survived or not (0 = No; 1 = Yes); this is our target
- `name` : A field rich in information as it contains title and family names
- `sex` : male/female
- `age` : Age, a significant portion of values are missing
- `sibsp` : Number of siblings/spouses aboard
- `parch` : Number of parents/children aboard
- `ticket` : Ticket number.
- `fare` : Passenger fare (British Pound).
- `cabin` : Does the location of the cabin influence chances of survival?
- `embarked` : Port of embarkation (C = Cherbourg; Q = Queenstown; S = Southampton)
- `boat` : Lifeboat, many missing values
- `body` : Body Identification Number
- `home.dest` : Home/destination



Classification using `rpart`

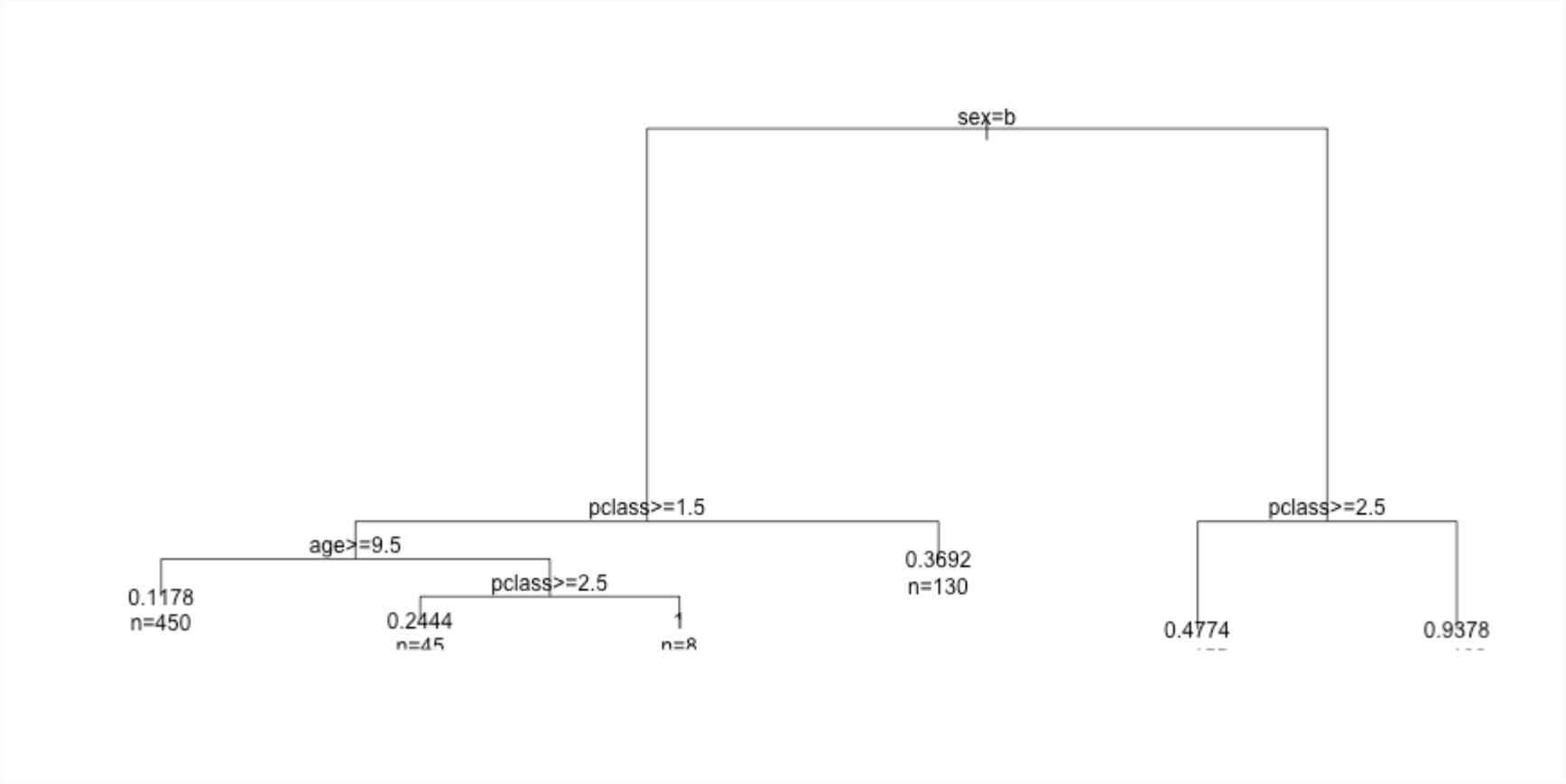
```
(titanic.rpart <- rpart(survived ~ pclass + sex + age + sibsp,
   data=titanic.train))
```

```
## n= 981
##
## node), split, n, deviance, yval
##       * denotes terminal node
##
##  1) root 981 231.651400 0.3822630
##    2) sex=male 633  97.251180 0.1895735
##      4) pclass>=1.5 503  61.693840 0.1431412
##        8) age>=9.5 450  46.757780 0.1177778 *
##        9) age< 9.5 53  12.188680 0.3584906
##         18) pclass>=2.5 45   8.311111 0.2444444 *
##         19) pclass< 2.5 8   0.000000 1.0000000 *
##      5) pclass< 1.5 130  30.276920 0.3692308 *
##    3) sex=female 348  68.146550 0.7327586
##      6) pclass>=2.5 155  38.670970 0.4774194 *
##      7) pclass< 2.5 193  11.253890 0.9378238 *
```



Classification using `rpart`

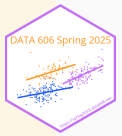
```
plot(titanic.rpart); text(titanic.rpart, use.n=TRUE, cex=1)
```



Classification using `ctree`

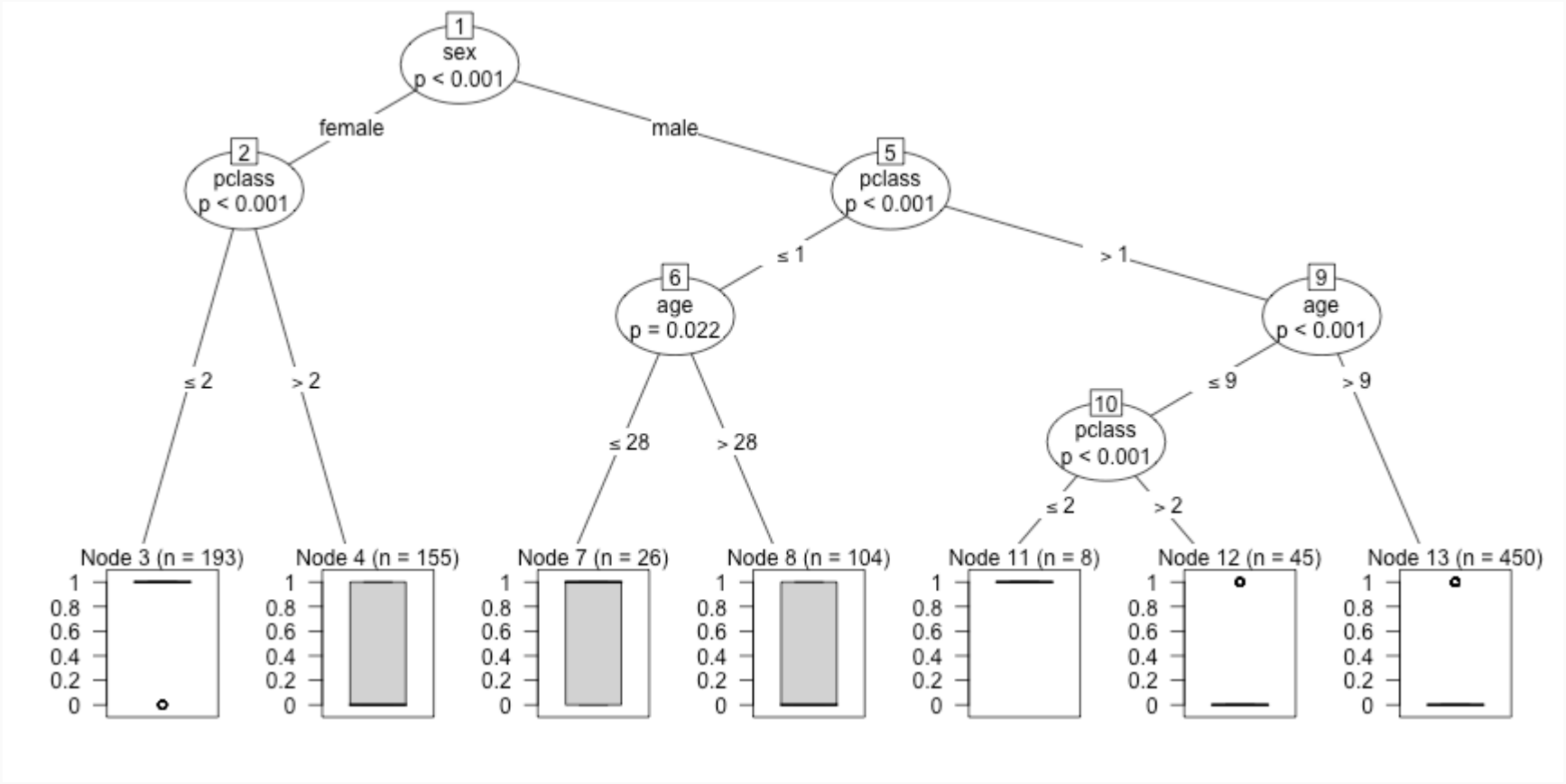
```
(titanic.ctree <- ctree(survived ~ pclass + sex + age + sibsp, data=titanic.train))
```

```
##
##      Conditional inference tree with 7 terminal nodes
##
## Response:  survived
## Inputs:  pclass, sex, age, sibsp
## Number of observations:  981
##
## 1) sex == {female}; criterion = 1, statistic = 280.286
##   2) pclass <= 2; criterion = 1, statistic = 83.503
##     3)*  weights = 193
##   2) pclass > 2
##     4)*  weights = 155
## 1) sex == {male}
##   5) pclass <= 1; criterion = 1, statistic = 22.768
##     6) age <= 28; criterion = 0.978, statistic = 7.679
##       7)*  weights = 26
##     6) age > 28
##       8)*  weights = 104
##   5) pclass > 1
##     9) age <= 9; criterion = 1, statistic = 17.781
##       10) pclass <= 2; criterion = 1, statistic = 16.543
```



Classification using `ctree`

```
plot(titanic.ctree)
```



Model Performance

The use of statistical models to predict outcomes, typically on new data, is called predictive modeling. Logistic regression is a common statistical procedure used for prediction. We will utilize a **confusion matrix** to evaluate accuracy of the predictions.

		True condition				
	Total population	Condition positive	Condition negative	Prevalence = $\frac{\Sigma \text{ Condition positive}}{\Sigma \text{ Total population}}$	Accuracy (ACC) = $\frac{\Sigma \text{ True positive} + \Sigma \text{ True negative}}{\Sigma \text{ Total population}}$	
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error	Positive predictive value (PPV), Precision = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Predicted condition positive}}$	
	Predicted condition negative	**False negative**, Type II error	**True negative**	False omission rate (FOR) = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Predicted condition negative}}$	
		True positive rate (TPR), Recall, Sensitivity, probability of detection, Power = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Condition negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR−}}$	F_1 score = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
		False negative rate (FNR), Miss rate = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Condition positive}}$	Specificity (SPC), Selectivity, True negative rate (TNR) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Condition negative}}$	Negative likelihood ratio (LR−) = $\frac{\text{FNR}}{\text{TNR}}$		

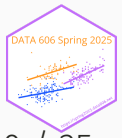
Predicting Heart Attacks

Source: https://www.kaggle.com/datasets/imnikhilanand/heart-attack-prediction?select=data.csv

```
heart <- read.csv('../course_data/heart_attack_predictions.csv')
heart <- heart |>
    mutate_if(is.character, as.numeric) |>
    select(!c(slope, ca, thal))
str(heart)
```

```
## 'data.frame':    294 obs. of  11 variables:
##  $ age     : int  28 29 29 30 31 32 32 32 33 34 ...
##  $ sex     : int  1 1 1 0 0 0 1 1 1 0 ...
##  $ cp      : int  2 2 2 1 2 2 2 2 3 2 ...
##  $ trestbps: num  130 120 140 170 100 105 110 125 120 130 ...
##  $ chol    : num  132 243 NA 237 219 198 225 254 298 161 ...
##  $ fbs     : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ restecg : num  2 0 0 1 1 0 0 0 0 0 ...
##  $ thalach : num  185 160 170 170 150 165 184 155 185 190 ...
##  $ exang   : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ oldpeak : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ num     : int  0 0 0 0 0 0 0 0 0 0 ...
```

Note: `num` is the diagnosis of heart disease (angiographic disease status) (i.e. Value 0: < 50% diameter narrowing -- Value 1: > 50% diameter narrowing)



Missing Data

We will save this for another day...

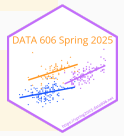
```
complete.cases(heart) |> table()
```

```
##
## FALSE  TRUE
##    33   261
```

```
mice_out <- mice::mice(heart, m = 1)
```

```
##
##  iter imp variable
##    1   1  trestbps  chol  fbs  restecg  thalach  exang
##    2   1  trestbps  chol  fbs  restecg  thalach  exang
##    3   1  trestbps  chol  fbs  restecg  thalach  exang
##    4   1  trestbps  chol  fbs  restecg  thalach  exang
##    5   1  trestbps  chol  fbs  restecg  thalach  exang
```

```
heart <- mice::complete(mice_out)
```



Data Setup

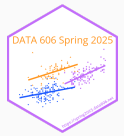
We will split the data into a training set (70% of observations) and validation set (30%).

```
train.rows <- sample(nrow(heart), nrow(heart) * .7)
heart_train <- heart[train.rows,]
heart_test <- heart[-train.rows,]
```

This is the proportions of survivors and defines what our "guessing" rate is. That is, if we guessed no one had a heart attack, we would be correct 62% of the time.

```
(heart_attack <- table(heart_train$num) %>% prop.table)
```

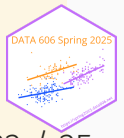
```
##
##         0         1
## 0.6341463 0.3658537
```



Model Training

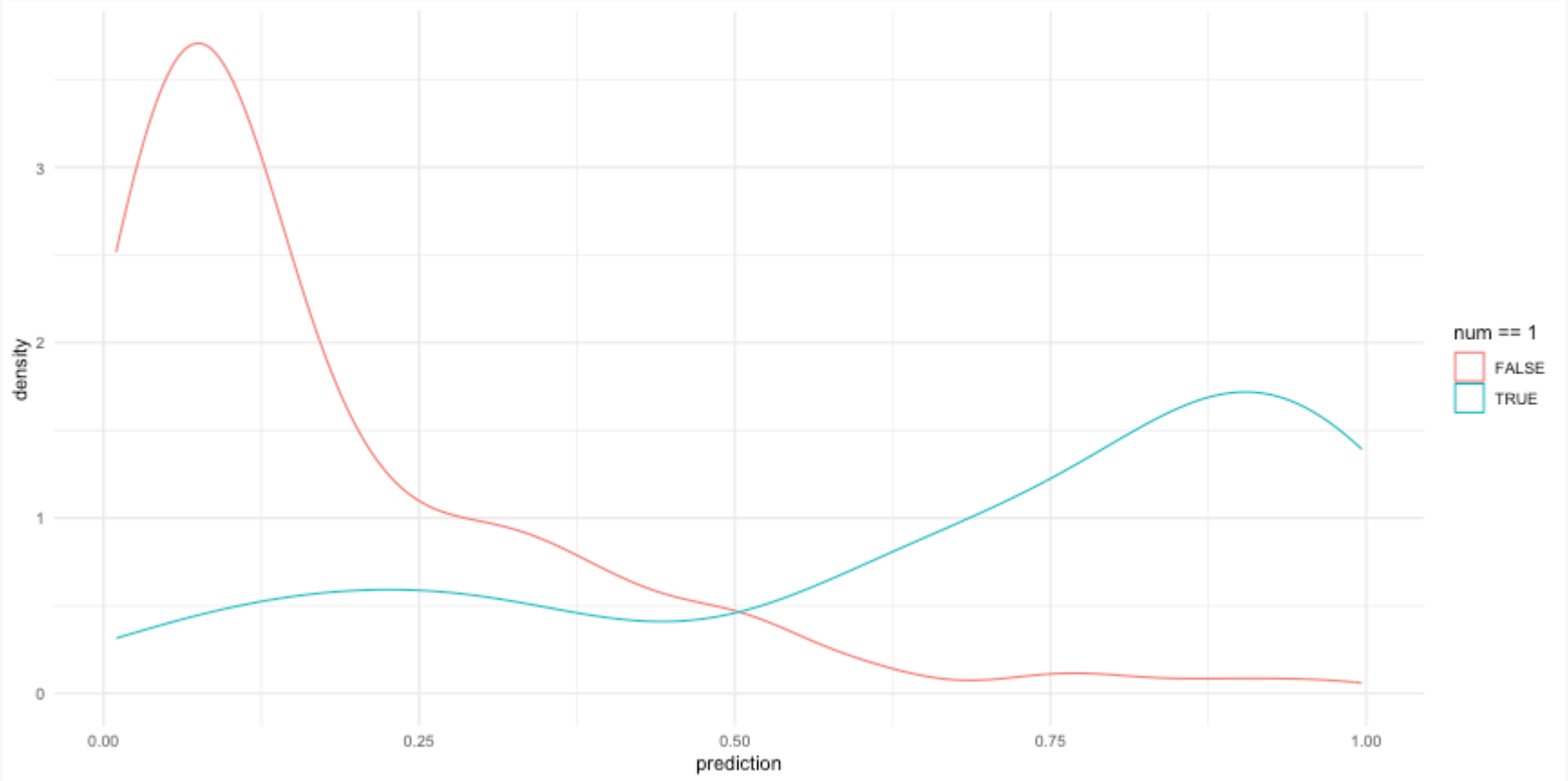
```
lr.out <- glm(num ~ ., data=heart_train, family=binomial(link = 'logit'))
summary(lr.out)
```

```
##
## Call:
## glm(formula = num ~ ., family = binomial(link = "logit"), data = heart_train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.779574   3.332608  -1.134  0.25674
## age         -0.007867   0.030622  -0.257  0.79724
## sex          1.413251   0.558548   2.530  0.01140 *
## cp           0.686401   0.243487   2.819  0.00482 **
## trestbps    -0.007194   0.012626  -0.570  0.56883
## chol         0.004791   0.002923   1.639  0.10118
## fbs          1.491788   0.834467   1.788  0.07382 .
## restecg     -0.741731   0.548210  -1.353  0.17605
## thalach     -0.007499   0.010417  -0.720  0.47158
## exang        1.156243   0.517161   2.236  0.02537 *
## oldpeak      1.222038   0.313464   3.898 9.68e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 269.25  on 204  degrees of freedom
## Residual deviance: 155.93  on 194  degrees of freedom
## AIC: 177.93
##
```



Predicted Values

```
heart_train$prediction <- predict(lr.out, type = 'response', newdata = heart_train)
ggplot(heart_train, aes(x = prediction, color = num == 1)) + geom_density()
```

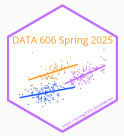


Results

```
heart_train$prediction_class <- heart_train$prediction > 0.5
tab <- table(heart_train$prediction_class,
             heart_train$num) %>% prop.table() %>% print()
```

```
##
##                    0          1
##    FALSE 0.58536585 0.09268293
##    TRUE  0.04878049 0.27317073
```

For the training set, the overall accuracy is 85.85%. Recall that 63.41% people did not have a heart attach. Therefore, the simplest model would be to predict that no one had a heart attack, which would mean we would be correct 63.41% of the time. Therefore, our prediction model is 22.44% better than guessing.



Checking with the validation dataset

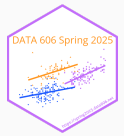
```
(survived_test <- table(heart_test$num) %>% prop.table())
```

```
##
##         0         1
## 0.6516854 0.3483146
```

```
heart_test$prediction <- predict(lr.out, newdata = heart_test, type = 'response')
heart_test$prediciton_class <- heart_test$prediction > 0.5
tab_test <- table(heart_test$prediciton_class, heart_test$num) %>%
    prop.table() %>% print()
```

```
##
##                   0          1
##    FALSE 0.59550562 0.08988764
##    TRUE  0.05617978 0.25842697
```

The overall accuracy is 85.39%, or 20.2% better than guessing.

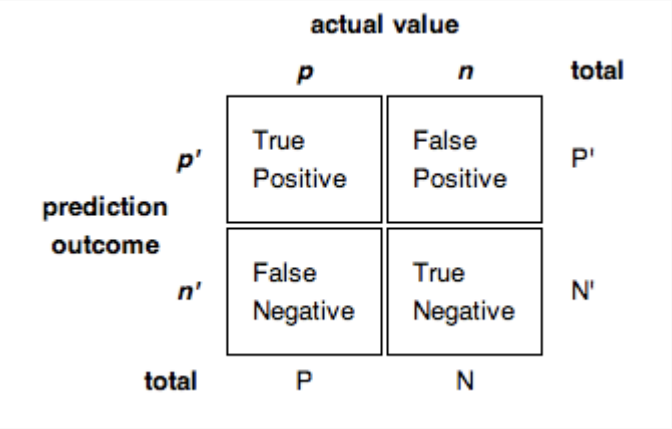


Receiver Operating Characteristic (ROC) Curve

The ROC curve is created by plotting the true positive rate (TPR; AKA sensitivity) against the false positive rate (FPR; AKA probability of false alarm) at various threshold settings.

In a classification model, outcomes are either as positive (*p*) or negative (*n*). There are then four possible outcomes:

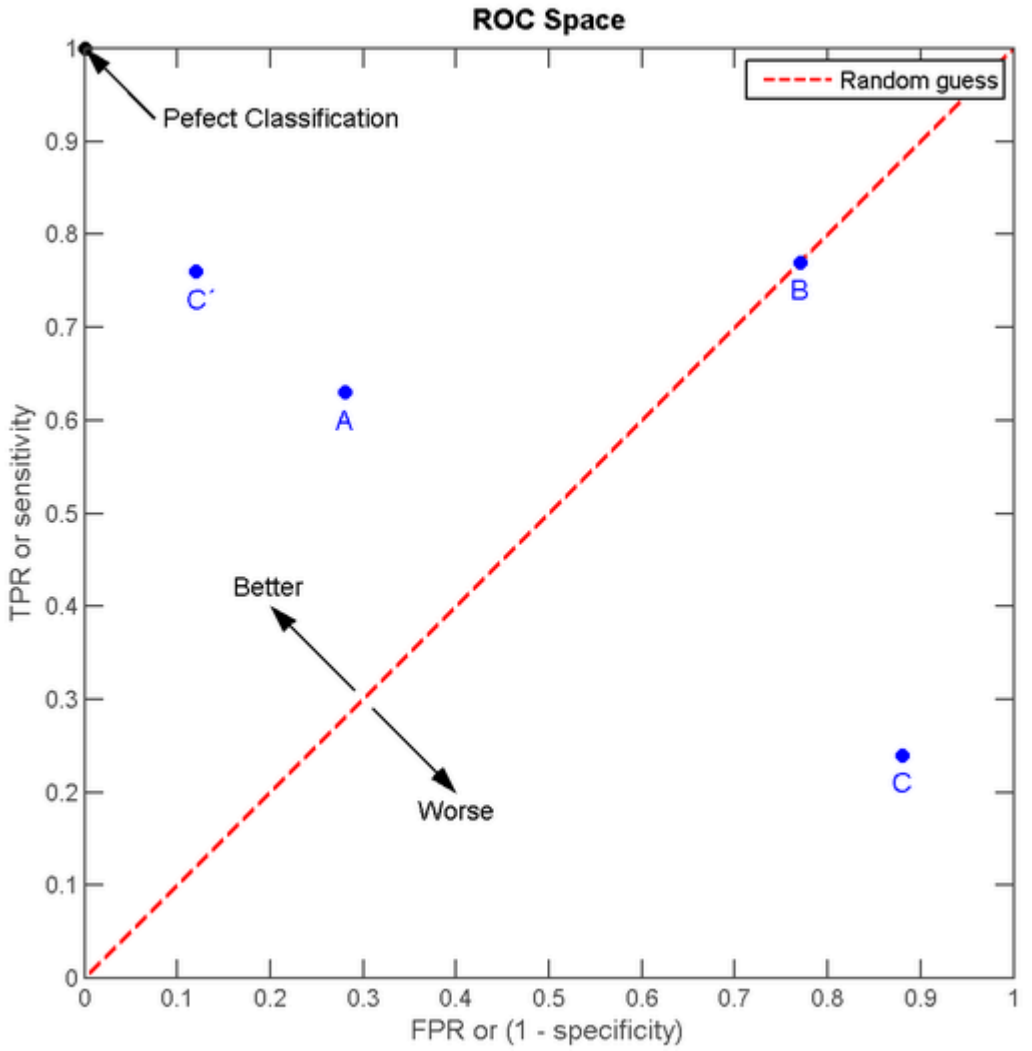
- **true positive** (TP) The outcome from a prediction is *p* and the actual value is also *p*.
- **false positive** (FP) The actual value is *n*.
- **true negative** (TN) Both the prediction outcome and the actual value are *n*.
- **false negative** (FN) The prediction outcome is *n* while the actual value is *p*.




```
roc <- calculate_roc(heart_train$prediction,
                     heart_train$num == 1)
summary(roc)
```

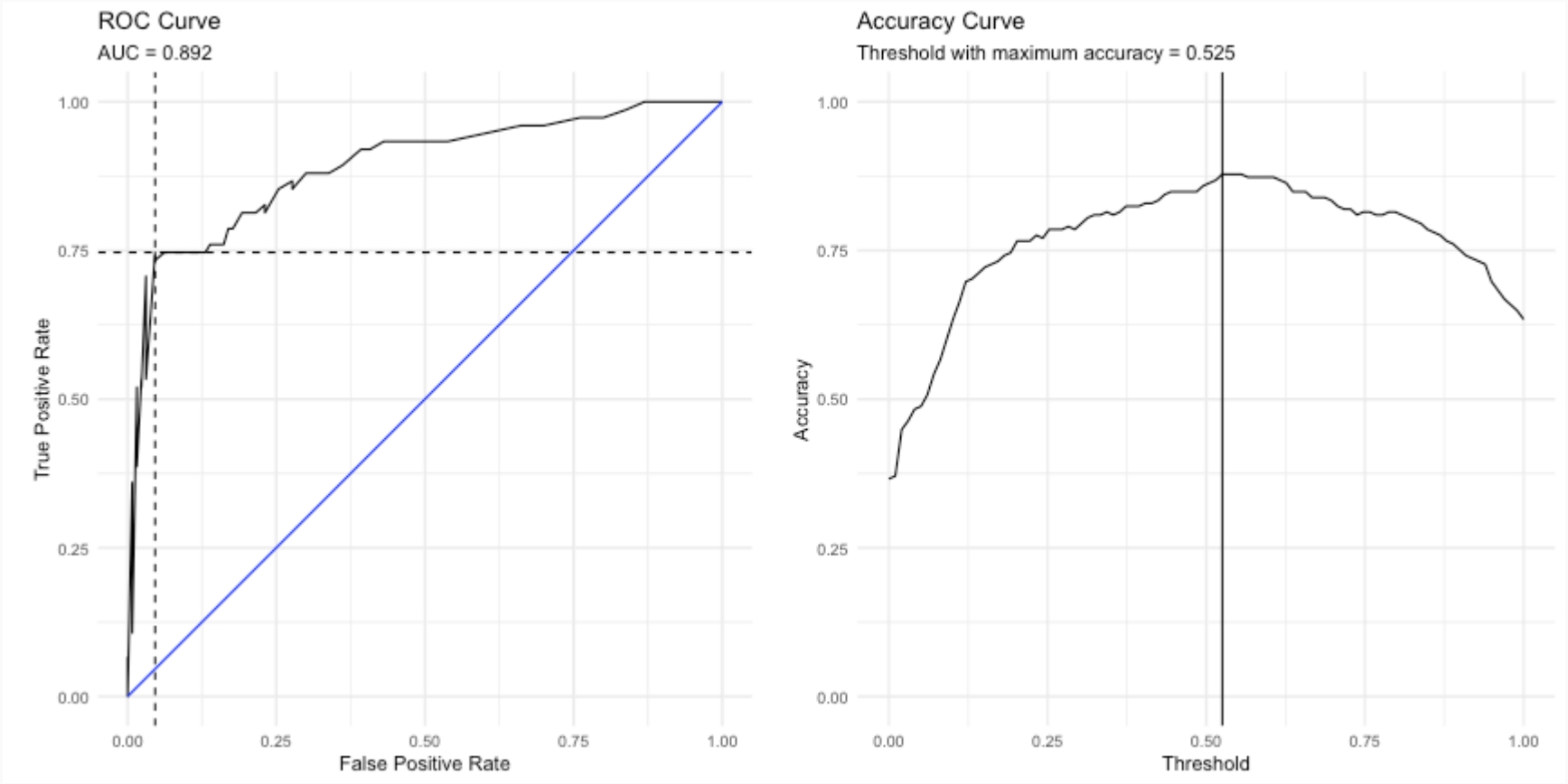
```
## AUC = 0.892
## Cost of false-positive = 1
## Cost of false-negative = 1
## Threshold with minimum cost = 0.525
```

ROC Curve


ROC Space
Random guess
Pefect Classification
C´
B
A
Better
Worse
C
TPR or sensitivity
FPR or (1 - specificity)
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1

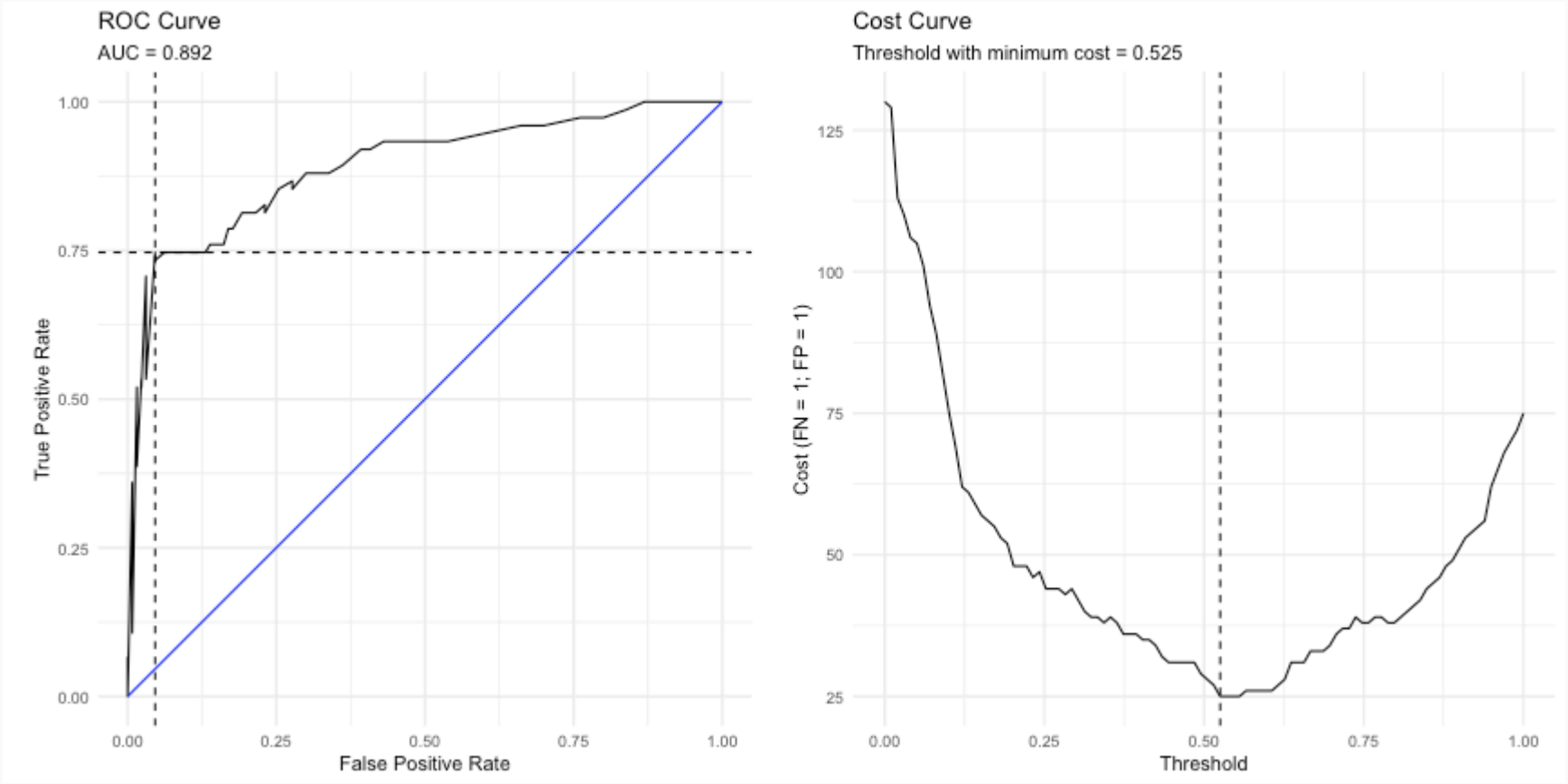
ROC Curve

```
plot(roc, curve = 'accuracy')
```



ROC Curve

```
plot(roc, curve = 'cost')
```

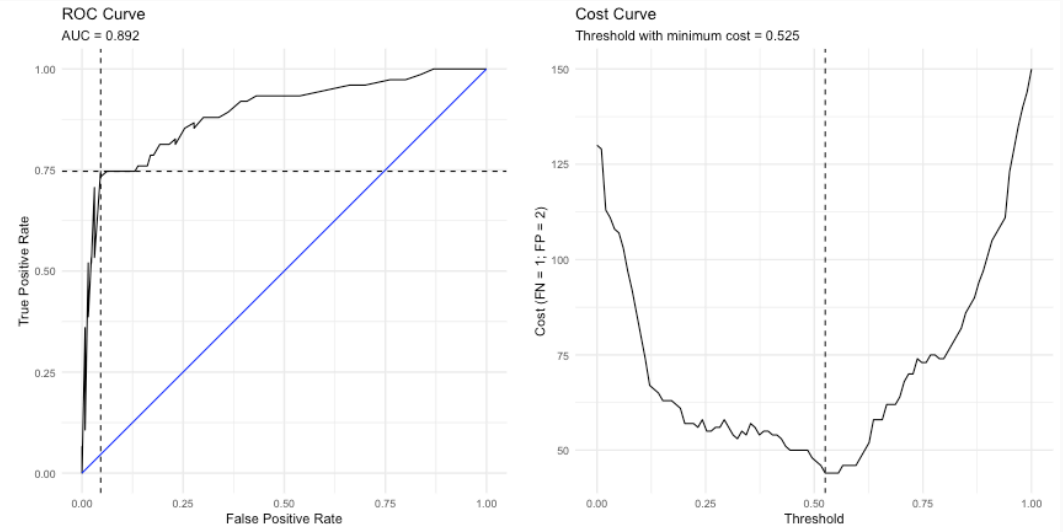


ROC Curve (false-negative up-weighted)

```
roc2 <- calculate_roc(
    heart_train$prediction,
    heart_train$num == 1,
    cost_of_fn = 2,
    cost_of_fp = 1)
summary(roc2)
```

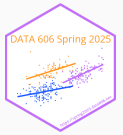
```
## AUC = 0.892
## Cost of false-positive = 1
## Cost of false-negative = 2
## Threshold with minimum cost = 0.525
```

```
plot(roc2, curve = 'cost')
```



Caution on Interpreting Accuracy

- Loh, Sooo, and Zing (2016) predicted sexual orientation based on Facebook Status.
- They reported model accuracies of approximately 90% using SVM, logistic regression and/or random forest methods.
- Gallup (2018) poll estimates that 4.5% of the Americal population identifies as LGBTG+.
- *My proposed model:* I predict all Americans are heterosexual.
- The accuracy of my model is 95.5%, or *5.5% better than Facebook's model!*
- Predicting "rare" events (i.e. when the proportion of one of the two outcomes large) is difficult and requires independent (predictor) variables that strongly associated with the dependent (outcome) variable.



Fitted Values Revisited

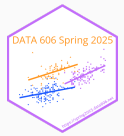
What happens when the ratio of true-to-false increases (i.e. want to predict "rare" events)?

Let's simulate a dataset where the ratio of true-to-false is 10-to-1. We can also define the distribution of the dependent variable. Here, there is moderate separation in the distributions.

```
test.df2 <- getSimulatedData(
    treat.mean=.6, control.mean=.4)
```

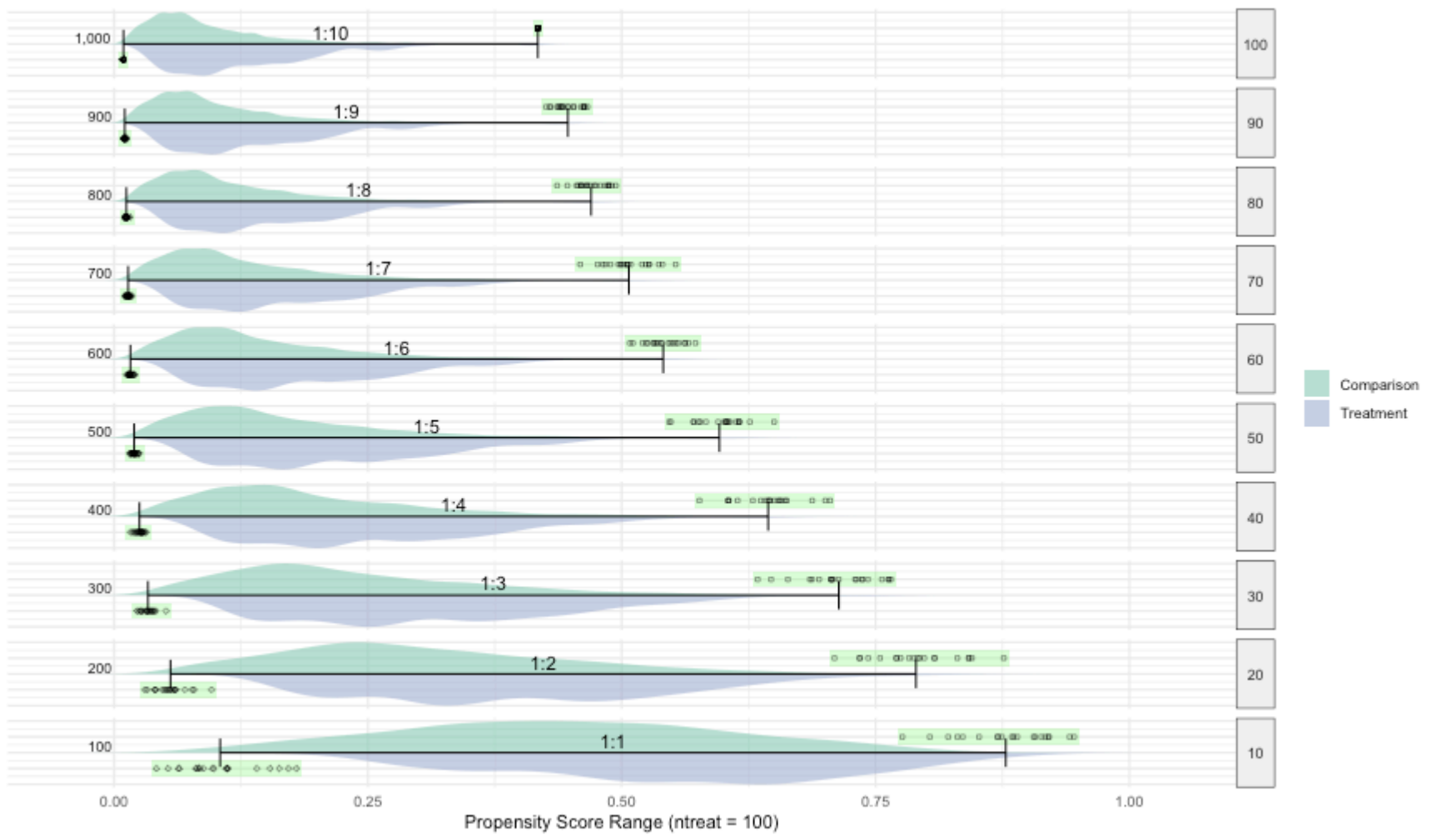
The `multilevelPSA::psrange` function will sample with varying ratios from 1:10 to 1:1. It takes multiple samples and averages the ranges and distributions of the fitted values from logistic regression.

```
psranges2 <- psrange(test.df2, test.df2$treat, treat ~ .,
                     samples=seq(100,1000,by=100), nboot=20)
```



Fitted Values Revisited (cont.)

```
plot(psranges2)
```



One Minute Paper

1. What was the most important thing you learned during this class?
2. What important question remains unanswered for you?

Scan the QR code or click here to complete the One Minute Paper